
APACHE AIRFLOW 3.1.7



APPENDIX A

Anti-patterns Troubleshooting

&

10 anti-pattern phổ biến, lỗi thường gặp & tối ưu hiệu năng

Target:	Backend / Distributed Systems Engineer
Version:	Apache Airflow 3.1.7
Type:	Quick reference guide
Use:	Tra cứu khi gặp vấn đề

Mục lục

1. Top 10 Anti-patterns

2. Common Errors & Fixes

3. Performance Tuning Checklist

1. Top 10 Anti-patterns

Anti-patterns là cách viết code "có vẻ đúng" nhưng gây ra vấn đề về sau. Đây là 10 anti-pattern phổ biến nhất trong Airflow và cách khắc phục:

Anti-pattern 1: Import pandas/spark ở top-level DAG

Hậu quả: Scheduler parse chậm, tiêu tốn nhiều RAM.

Fix: Lazy import trong task

```
def process_data(**context):
    import pandas as pd # Import bên trong hàm
    df = pd.read_csv(...)
    return df.to_dict()
```

Anti-pattern 2: XCom chứa data lớn

Hậu quả: DB bloated, query chậm, có thể crash scheduler.

Fix: Lưu Redis, XCom chỉ lưu key

```
def process(**context):
    import redis
    r = redis.Redis(host='redis', db=1)
    data_key = f"xcom:{context['run_id']}:result"
    r.set(data_key, large_data_json, ex=3600) # TTL 1 giờ
    return {"redis_key": data_key} # Chỉ truyền key
```

Anti-pattern 3: Catchup=True mặc định

Hậu quả: Chạy lại hàng trăm historical runs cùng lúc, quá tải hệ thống.

Fix: Luôn đặt `catchup=False`

```
with DAG(..., catchup=False) as dag: # Bắt buộc cho production
```

Anti-pattern 4: Sensor poke mode không timeout

Hậu quả: Worker slot bị chiếm mãi, không task nào được chạy.

Fix: Dùng `mode='reschedule'` hoặc `deferrable=True`

```
FileSensor(task_id='wait', filepath='/file.txt',  
            mode='reschedule', timeout=timedelta(hours=2))
```

Anti-pattern 5: Chạy heavy computation trong worker

Hậu quả: Worker bị kill, task bị mark failed không rõ nguyên nhân.

Fix: Submit Spark/K8s job, worker chỉ orchestrate

```
SparkSubmitOperator(task_id='heavy_job', application='job.py')
```

Anti-pattern 6: Hardcode credentials

Hậu quả: Lộ credentials qua git, security risk nghiêm trọng.

Fix: Dùng Airflow Connections + Vault

```
hook = PostgresHook(postgres_conn_id='my_conn') # Không hardcode
```

Anti-pattern 7: Không đặt retries

Hậu quả: Task fail 1 lần (network glitch) = pipeline fail.

Fix: Luôn có retries và retry_delay

```
default_args={'retries': 3, 'retry_delay': timedelta(minutes=5),
              'retry_exponential_backoff': True}
```

Anti-pattern 8: Dùng Variable trong top-level DAG

Hậu quả: Query DB mỗi lần parse DAG — chậm và không cần thiết.

Fix: Dùng Jinja template {{ var.value.x }}

```
BashOperator(task_id='t', bash_command='echo "{{{{ var.value.env }}}}"')
```

Anti-pattern 9: DAG quá lớn (>100 tasks)

Hậu quả: Khó debug, khó maintain, UI chậm.

Fix: Split thành sub-DAGs hoặc TaskGroup

```
with TaskGroup("group1", dag=dag) as g1:
    task1 >> task2 >> task3
```

Anti-pattern 10: Không monitor scheduler lag

Hậu quả: DAG chạy trễ mà không biết, mất dữ liệu hoặc vi phạm SLA.

Fix: Prometheus + alert tên scheduler_heartbeat

```
# Alert rule
- alert: AirflowSchedulerDown
  expr: airflow_scheduler_heartbeat == 0
```

2. Common Errors & Fixes

Bảng 1: Lỗi thường gặp và cách khắc phục

Error	Nguyên nhân	Cách fix
<code>SchedulerHeartbeatException</code>	Scheduler quá tải hoặc DB chậm	Scale scheduler, tối ưu DB, check network
<code>Executor has no open slots</code>	Worker đầy	Scale worker, tăng parallelism
<code>XCom too large</code>	Push data lớn qua XCom	Dùng Redis backend
<code>DAG not found</code>	File chưa parse	Check dags_folder, fileloc, scheduler logs
<code>Task instance not found</code>	Race condition hoặc DB inconsistent	Không xóa DB thủ công, dùng airflow db reset cẩn thận
<code>Connection refused</code>	Service chưa start	Check docker compose healthcheck, depends_on

3. Performance Tuning Checklist

Checklist để tối ưu hiệu năng Airflow production:

- **Scheduler parsing:** `AIRFLOW__SCHEDULER__PARSING_PROCESSES` = số CPU core / 2
- **DAG refresh:** `AIRFLOW__SCHEDULER__DAG_DIR_LIST_INTERVAL` $\geq$ 120s nếu DAG ít thay đổi
- **Concurrency:** `AIRFLOW__CORE__PARALLELISM` $\leq$ tổng worker slots
- **DB pool:** `AIRFLOW__DATABASE__SQL_ALCHEMY_POOL_SIZE` $\geq$ 10
- **Serialization:** Dùng DAG serialization để giảm parse time
- **Schedule caching:** Enable `schedule_interval` caching cho DAGs ổn định
- **Sensors:** Dùng `deferrable` thay vì `poke` mode
- **XCom:** XCom backend = Redis cho data > 1KB
- **Task duration:** Task nên < 1 giờ (nếu dài hơn, cân nhắc split)
- **Logs:** Remote logging = True → Redis/CloudWatch (không để log đầy disk)

- **Idempotent:** Pipeline phải idempotent — chạy lại nhiều lần không sai
- **Pools:** Dùng pools để giới hạn task parallelism cho resource hạn chế

Debug flow

Khi gặp lỗi, debug theo thứ tự: (1) Đọc error message kỹ, (2) Check task logs trong UI, (3) Kiểm tra scheduler logs, (4) Query metadata DB, (5) Test task độc lập bằng `airflow tasks test`.